

Assignment . 7

Title:

Implement a generic program using any collection class to count the number of elements in a collection that have a specific property such as even numbers, odd number, prime number and palindromes.

Aim: Template and Generic Programming

Objectives: To learn the concept templates and generic programming

Theory:

Generic Types

- ✓ Generic type represents classes, interfaces and methods in a type safe manner
- ✓ Generic types can act on any type of data
- ✓ All Generic types are subclasses of Object class, it acts on Objects only
- ✓ Generic types act on advanced data type only
- ✓ It is not possible to create an object to Generic type itself
- ✓ Using generic types, we can avoid casting in many cases

When we create a class with an instance variable to store an Integer object, it can be used to store Integer type data only

We cannot use that instance variable to store a Float class object or a String type Object To store different types of data into a class, we have to write the same class again and again by changing the data type of the variables This can be avoided using a generic class A generic class represents a class that is type-safe This means a generic class can act upon any data type Generic classes and generic interfaces are also called „parameterized types“ because they use a parameter that determines which data type they should work upon

Generic Method: We can make a method alone as generic method by writing the generic parameter before the method return type as:

```
returntype methodname ()
```

```
{  
Method code;  
}
```

eg: void display_data () { Method body; }

Generic Interface:

It is possible to develop an interface using generic type concept. The general form of generic interface looks like:

```
interface interface_name
```

```
{ //method that accepts any object return_type method_name ( T  
  object_name ); } Here, T represents any data type which is used in the  
interface.
```

We can write an implementation class for the above interface as: class class_name implements interface_name

```
{ public return_type method_name ( T object_name )  
  { //provide body of the method }  
}
```

Input: count variable

Output: count elements in any collection with a specific property,

Steps:

Step 1: Start

Step 2: Define the Interface: Start by defining a generic interface that represents the property you want to check. You can use a functional interface in Java, for example, to represent a condition that checks if an element meets a specific property.

Step 3: Implement Property Check Functions: Next, implement functions that check for the specific properties you're interested in. Here's an example of functions to check for even and odd numbers, prime numbers, and palindromes.

Step 4: Create a Generic Counting Method: Now, create a generic method that takes a collection and a property checker function as parameters. The method iterates through the collection and counts the elements that meet the specified property.

Step 5: Use the Generic Method: You can use the generic countElementsWithProperty method with various collection classes and property checkers

Conclusion:

Thus we have implemented generic programming concept using java.

Questions:

1. What Is a Generic Type Parameter
2. What Are Some Advantages of Using Generic Types
3. If a Generic Type Is Omitted When Instantiating an Object, Will the Code Still Compile
4. How Does a Generic Method Differ from a Generic Type?
5. What Is Type Inference?
6. What Is a Bounded Type Parameter

7. Is It Possible to Declared a Multiple Bounded Type Parameter?
8. What Is a Wildcard Type

(write answer for the above questions) & Attached code with output